

Multi-Source Candidate Data Transformer

Eightfold - Engineering Intern Assignment Design Document

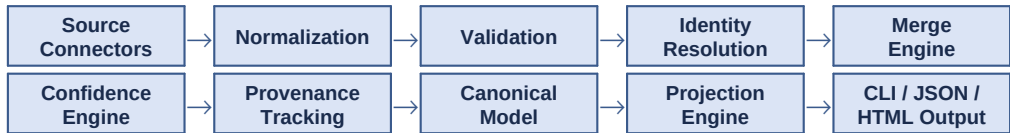
vikassharma53489@gmail.com • github.com/Vikas1715 • linkedin.com/in/vikas1715v

Vikas Sharma

Project Overview

A CLI-first pipeline that consolidates candidate data from a structured source (CSV/ATS JSON), a resume, recruiter notes, and the GitHub API into one canonical profile. Every field carries full source provenance and a deterministic confidence score, and conflicts between sources are resolved by explicit trust rules rather than heuristics, ML, or fuzzy matching. Built for auditability first — every output value can be traced back to exactly where it came from and why it won.

High-Level Architecture



Four inputs (structured file, resume, notes, GitHub) are fetched concurrently via `ThreadPoolExecutor`, fully synchronized before merge — so parallel I/O never affects determinism.

Core Components

Module	Responsibility
Source Connectors	Pluggable readers (CSV/JSON, PDF/TXT resume, notes, GitHub REST) — new source = new subclass, no pipeline change
Normalization	Per-field pure functions: email/phone/name/list formatting, applied independently per source
Validation	Field-level (regex/range) + cross-field consistency checks (e.g. seniority vs. years)
Identity Resolution	Deterministic hierarchy: explicit ID → email → phone → GitHub username → unresolved. No fuzzy matching
Merge Engine	Per-field conflict resolution by source trust + precedence; losing values kept as evidence, never discarded
Confidence Engine	Additive evidence model: validation, corroboration, trust, completeness, freshness, minus conflict penalty
Projection Engine	Maps internal canonical model to flat output schema; validates shape before write
Reports	Data Quality Report, Metrics Report, and an HTML explainability report per candidate
GitHub Cache	File-based, TTL-enforced, atomic writes — cuts repeat API calls and rate-limit risk

Key Design Decisions

Deterministic processing

No ML, no fuzzy logic, no randomness anywhere in identity, merge, or scoring — identical input always yields identical output (verified by repeated-run diff).

Provenance tracking

Every field keeps raw value, normalized value, source, method, validity and timestamp — nothing is ever lost, even from losing sources.

Confidence engine

Additive evidence scoring (not a weighted average) — corroboration and conflict penalties are signals a plain average can't express.

Scalability & Extensibility

Candidates share no mutable state, so `run_single_candidate()` drops directly into a queue worker with no redesign. Batch mode today runs sequentially on one machine; the same function scales horizontally by running more workers against the same manifest. The only shared resource — the GitHub cache — is already file-based and isolated in one module, the sole seam that would move to Redis/S3 in a distributed setup.

Modular connectors

Each source implements one `SourceConnector.fetch()` method; adding DOCX/XML/LinkedIn later touches zero existing code.

Conflict-aware merge

Disagreements are resolved by trust score + precedence and flagged, not silently overwritten; both values stay visible in the report.

GitHub API caching

Local TTL-based file cache keyed by username, atomic writes, graceful 403/timeout handling — avoids redundant calls and rate-limit failures.

Technologies Used

Python 3.9+, argparse (CLI), dataclasses (models), PyYAML (config), pypdf (resume PDF text), requests (GitHub REST API), `concurrent.futures.ThreadPoolExecutor` (parallel I/O), unittest (40 unit tests). No ML/statistics libraries — deliberately, per the assignment's determinism requirement.

Why This Design Is Reliable

Every decision the pipeline makes — which value won, why, and how confident it is — is reconstructable from the output alone, since nothing is discarded on merge and every stage is a pure, independently-tested function of its input. Correctness is backed by 40 unit tests across normalization, validation, identity, merge, confidence, projection, and caching, plus a manual determinism check confirming identical output across repeated runs of the same input. The pipeline is fully deterministic, explainable, and traceable through field-level provenance, ensuring every output can be independently verified and audited.